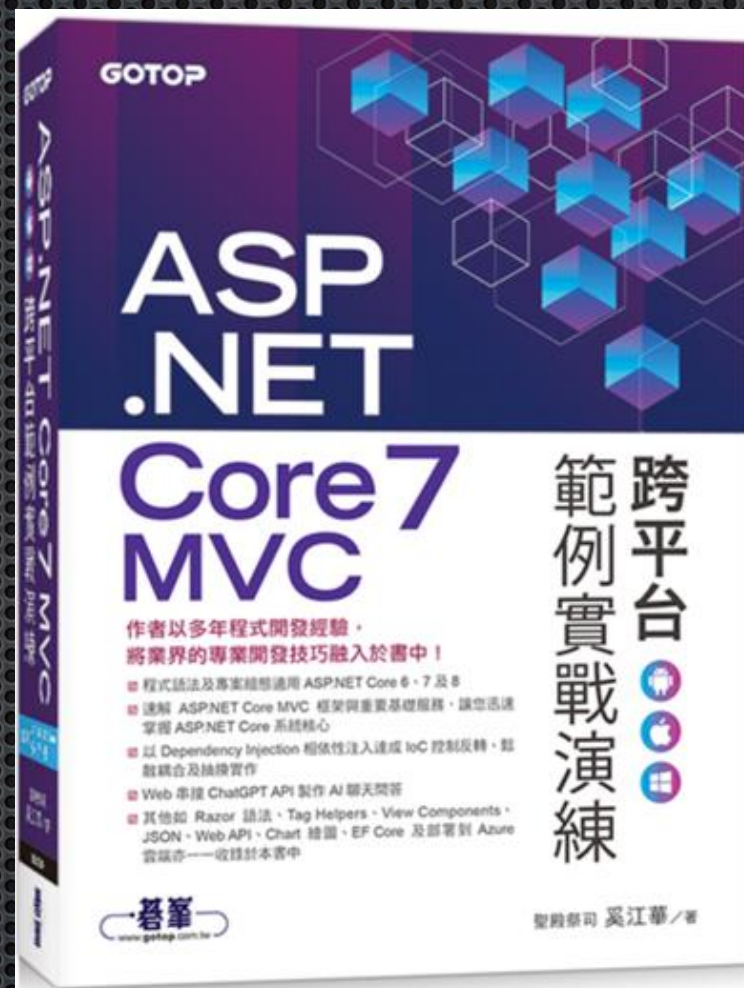


第15章

Entity Framework Core 資料庫存取與Transaction交易



作者：聖殿祭司。奚江華

大綱

15-1 Entity Framework Core與ORM概觀

15-2 Entity Framework 6.x的三種開發模式

15-3 設定 EF Core所需套件及資料庫連線

15-4 用Code First對既有資料庫Scaffolding出DbContext及
模型檔

15-5 Entity Framework Core 查詢資料庫常用語法

15-6 資料庫交易程式

Entity Framework Core與 ORM概觀

15-1

ORM(O/RM或O/R Mapping)

- ORM 是指 Object-Relational Mapping，它是Object物件和資料庫之間對映的一種技術
- Object 與資料庫之間的溝通是透過 ORM 軟體框架來處理(例如Entity Framework)
- 讓開發人員只需面對 Object 物件做 CRUD，剩下對後端資料庫的存取程式，ORM會替你處理掉，以節省瑣碎的資料庫存取程式撰寫

CRUD

- 對資料或物件的Create、Read、Update和Delete作業
- 新增、讀取、更新與刪除

ORM運作模式

- ORM負責後端資料庫存取，資料庫被抽象化
- 資料庫即使抽換成另一種資料庫平台，程式不受影響



EF Core運作模式

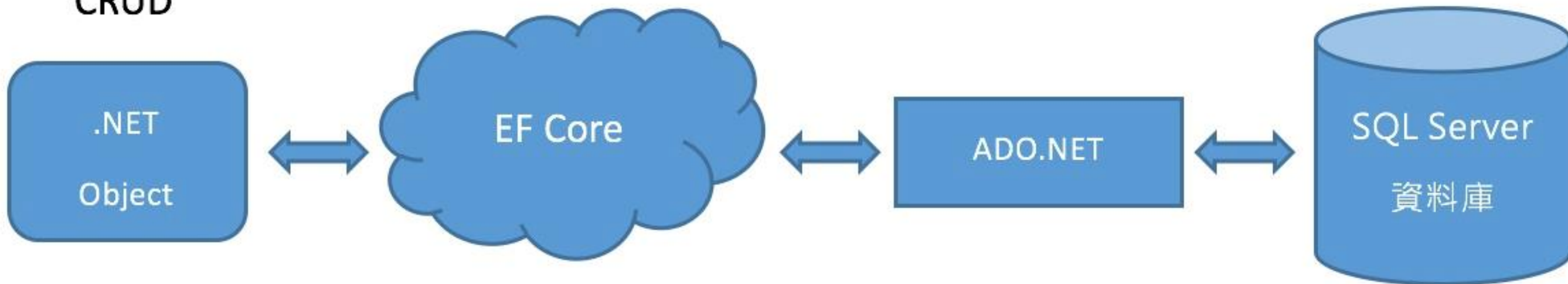
- EF Core是一種 O/R Mapping 的框架實作
- 只需對C#物件(DbContext)做CRUD操作，後續 EF Core會自動處理對資料庫的存取作業，而不必撰寫傳統 ADO.NET程式



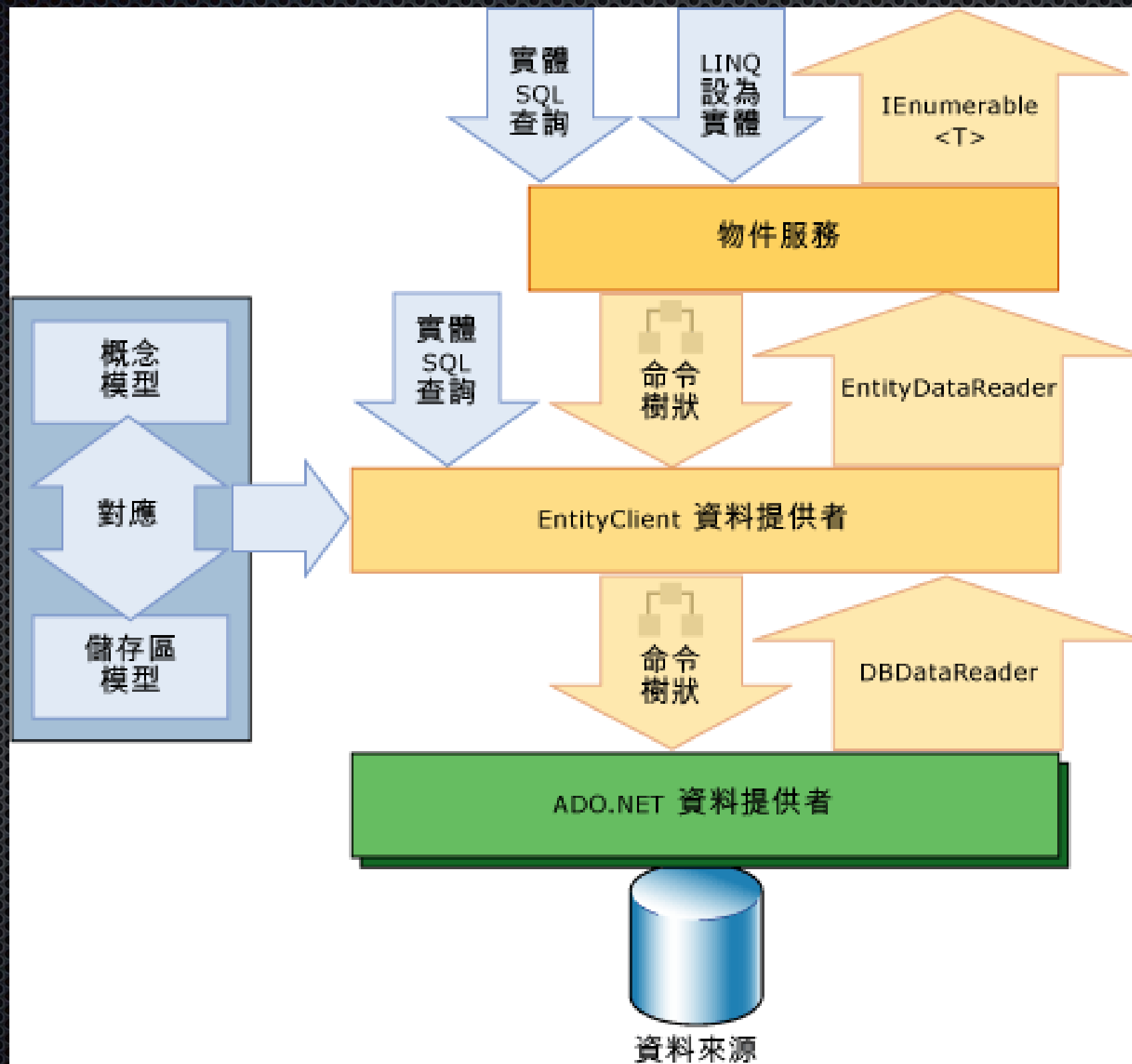
EF Core透過ADO.NET與資料庫作業(CRUD)

- EF和資料庫的溝通，底層仍是透過ADO.NET來進行

CRUD



EF細部架構圖



Entity Framework 6.x的 三種開發模式

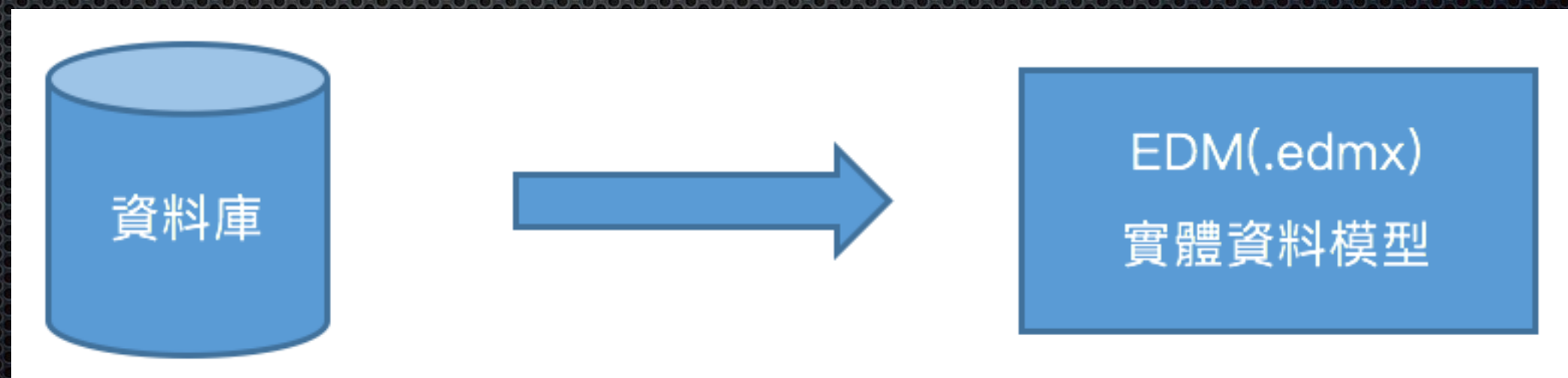
15-2

EF三種開發模式

- Database First 資料庫優先
- Model First 模型優先
- Code First 程式優先

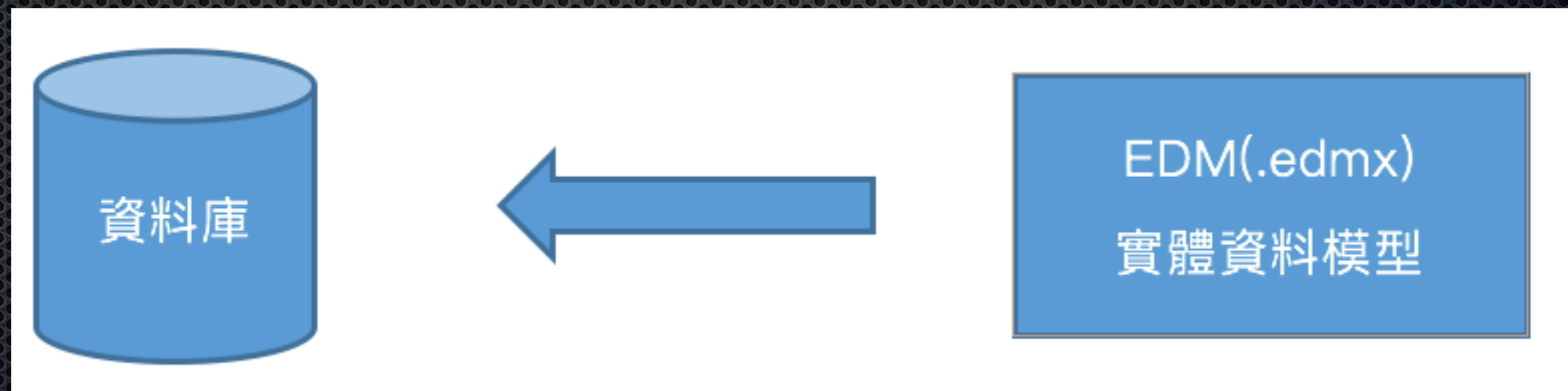
Database First 資料庫優先

- 以既有資料庫為優先考量，從資料庫產出 Entity Data Model 實體資料模型(.edmx)
- .edmx是定義 Entities實體和 Association 關聯性，同時EF也支援.edmx 的視覺化模型設計工具



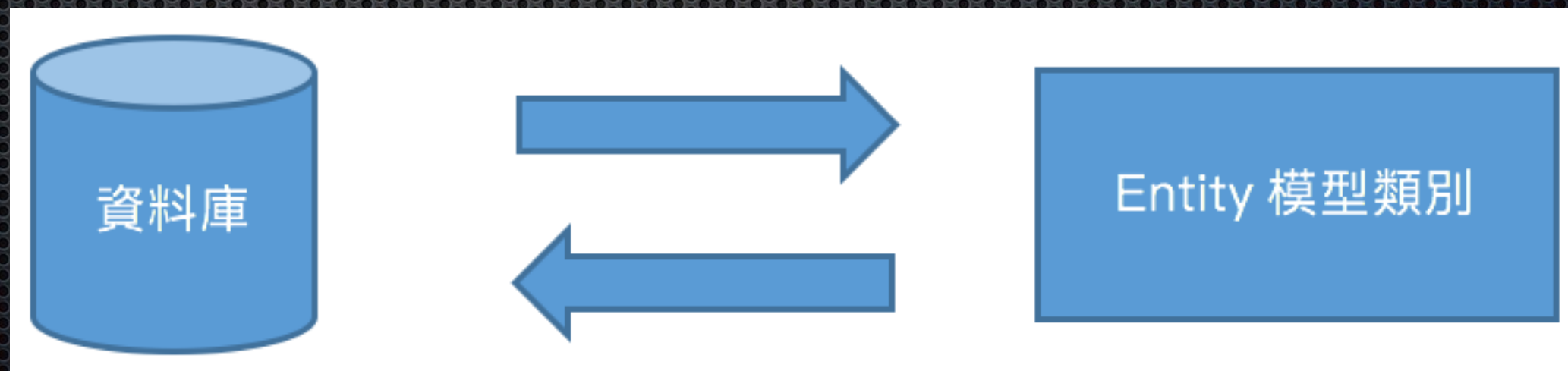
Model First 模型優先

- 以Model設計為優先考量，先設計好 Entity Data Model(.edmx)，再由 Model 產出新的資料庫
- 支援視覺化 模型設計工具



Code First 程式優先

- Code First是以程式撰寫Entity Class來代表Entity Data Model，沒有.edmx 檔
- 不支援視覺化模型設計工具
- **Code First 在微軟已定調是現在及未來發展的主流**



三種開發模式技術發展

- 以發展歷史來看，三種模式中最早出現的是 Database First 和 ModelFirst
- 後來 EF 4.1 才推出 Code First
- 然因 Database First 和 Model First 一些技術缺點和限制，故在 EF Core 只保留 Code First 開發模式

Code First作業方式

- 對既有資料庫產出 DbContext 及 Model 模型檔

例如對既有Northwind資料庫，產出 DbContext 及 Model 模型檔

- 建立 DbContext 及 Model 模型檔後產出對應新資料庫

新的專案沒有資料庫的情況下，可先建立 DbContext 及 Model 模型檔，再透過Code First Migration同步機制創建出資料庫 / 資料表 / 種子資料

設定 EF Core所需套件 及資料庫連線

15-3

使用 EF Core的前置工作

- ① 安裝 EF Core Tools CLI命令工具
- ② 安裝 EF Core 所需 NuGet 套件
- ③ 在 appsettings.json 設定資料庫連線
- ④ 建立對映資料庫的DbContext(視階段建立)

安裝 EF Core Tools CLI 命令工具

15-3-1

① 安裝 EF Core Tools CLI 命令工具

.NET Core SDK CLI 無內建支援 EF Core Tools 命令工具，須額外安裝，才能以CLI命令執行 Migrations，或從既有資料庫 scaffolding 產出程式

- 安裝指令

```
dotnet tool install --global dotnet-ef --version 7.0.4
```

- 查看版本

```
dotnet tool list -g
```

- 更新指令

```
dotnet tool update -g dotnet-ef
```


- 解除安裝指令

dotnet tool **uninstall** -g dotnet-ef

- 在 nuget.org 搜尋工具詳細說明及所有版本資訊

dotnet tool **search** dotnet-ef --detail

② 在NuGet 主控台安裝 EF Core 相關 NuGet 套件

```
install-package Microsoft.EntityFrameworkCore -Version 7.0.10
```

```
install-package Microsoft.EntityFrameworkCore.Tools -Version 7.0.10
```

```
install-package Microsoft.EntityFrameworkCore.SqlServer -Version 7.0.10
```

```
install-package Microsoft.EntityFrameworkCore.Design -Version 7.0.10
```

```
install-package Microsoft.VisualStudio.Web.CodeGeneration.Design -Version 7.0.9
```

```
Get-Package (列出已安裝套件)
```


在命令視窗用CLI命令安裝 EF Core 相關 NuGet 套件

```
dotnet add package Microsoft.EntityFrameworkCore --version 7.0.10
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools --version 7.0.10
```

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 7.0.10
```

```
dotnet add package Microsoft.EntityFrameworkCore.Design --version 7.0.10
```

```
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design --version 7.0.9
```

```
dotnet list package (列出已安裝套件)
```


③ appsettings.json資料庫連線

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*",  

```

設定資料庫連線

```
    "ConnectionStrings": {  
      "NorthwindConnection": "Server=(localdb)\\mssqllocaldb;Database=Northwind;  
                              Trusted_Connection=True;MultipleActiveResultSets=true"  
    }  
  }  
}
```


在開發環境以User Secret
(使用者祕密)建立資料庫連線

15-3-2

以使用者祕密保護敏感資訊

- 有時資料庫連線屬於敏感資訊，甚至不想提交到 GitHub 上，以避免連線資訊外洩
- ASP.NET Core 提供 User Secret (使用者祕密) 機制，讓你保存敏感資訊，而不需將這些資訊寫在 appsettings.json 組態檔

① 在專案中啟用在 User Secret

- 請先刪除appsettings.json中Northwind資料庫連線設定
- 在命令視窗切換到專案路徑，啟用 User Secret

dotnet user-secrets init

專案.csproj檔

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net7.0</TargetFramework>
    <Nullable>disable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>b0875ed9-6954-49d8-84e5-04937c3da88d</UserSecretsId>
  </PropertyGroup>
</Project>
```

使用者祕密Id設定

在 User Secret 中新增資料庫 連線設定

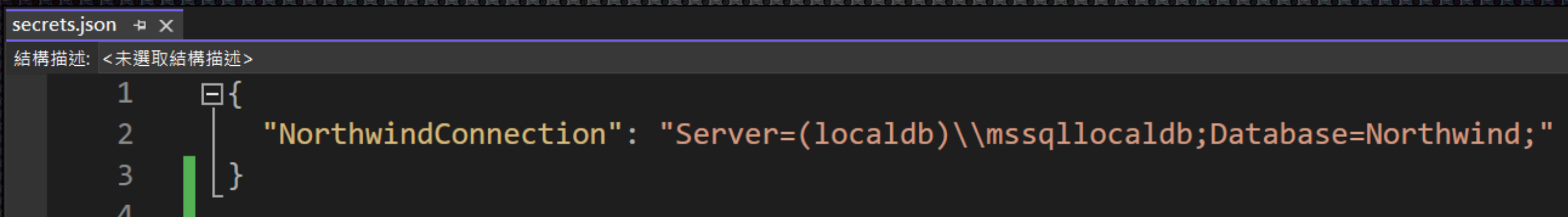
```
dotnet user-secrets set "NorthwindConnection"  
"Server=(localdb)\mssqllocaldb;Database=Northwind;"
```

或

```
dotnet user-secrets set NorthwindConnection  
"Server=(localdb)\mssqllocaldb;Database=Northwind;"
```


檢視User Secret 設定檔

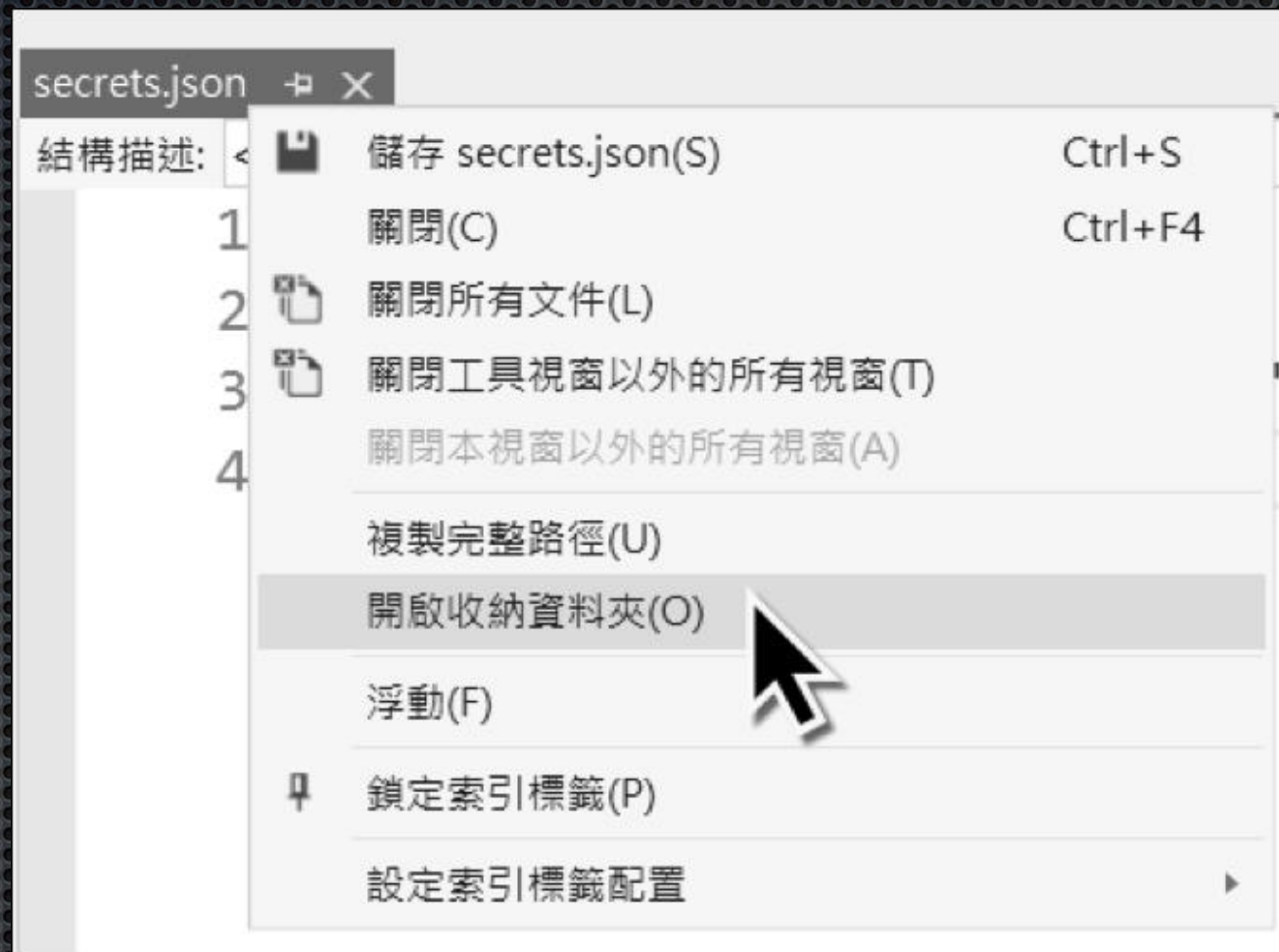
- 於Visual Studio在專案按滑鼠右鍵→【管理使用者密碼】，就會出現 secrets.json 檔



```
secrets.json  X
結構描述: <未選取結構描述>
1  {
2    "NorthwindConnection": "Server=(localdb)\\mssqllocaldb;Database=Northwind;"
3  }
4
```


檢視secrets.json檔案路徑

- 在 secrets.json 頁籤按滑鼠右鍵→【開啟收納資料夾】，檔案總管中就會顯示實際儲存路徑



C:\Users\apple\AppData\Roaming\Microsoft\UserSecrets\
9400669b-d7da-496a-9a5a-a46a6e5671c6\secrets.json

列出 / 删除使用者秘密

- 列出使用者秘密

```
dotnet user-secrets list
```

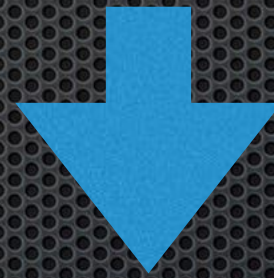
- 删除特定使用者秘密

```
dotnet user-secrets remove NorthwindConnection
```


EF Core資料庫連線要如何改成抓使用者祕密？

- 由於 secrets.json 也是載入集成到 Configuration 組態中，因此只需調整組態Key名稱

```
services.AddDbContext<CardContext>(options =>  
    options.UseSqlServer(Configuration.GetConnectionString("CardContext"));
```



```
services.AddDbContext<CardContext>(options =>  
    options.UseSqlServer(Configuration["NorthwindContext"]));
```


以程式讀取資料庫連線字串

15-3-3

在Controller讀取資料庫連線字串

```
public class EmployeesController : Controller
{
    private readonly NorthwindContext _context;

    public EmployeesController(NorthwindContext context)
    {
        _context = context;
    }

    public IActionResult GetConnectionString()
    {
        //讀取DbContext使用的資料庫連線
        string conn1 = _context.Database.GetConnectionString();
        string conn2 = _context.Database.GetDbConnection()
            .ConnectionString;

        ViewData["Conn"] = conn1;

        return View();
    }
}
```

讀取資料庫連線

在View讀取資料庫連線字串

```
@inject IConfiguration config
```

```
@{
```

```
    ViewData["Title"] = "GetConnectionString";
```

```
    //讀取appsettings.json資料庫連線字串
```

```
    string conn1 = config.GetConnectionString("NorthwindConnection");
```

```
    string conn2 = config["ConnectionStrings:NorthwindConnection"];
```

```
    string conn3 = config.GetValue<string>("ConnectionStrings:NorthwindConnection");
```

```
    string conn4 = config.GetSection("ConnectionStrings:NorthwindConnection").Value;
```

```
    string conn5 = config.GetSection("ConnectionStrings")["NorthwindConnection"];
```

```
}
```

```
<p>@conn1</p>
```

```
<p>@conn2</p>
```

```
<p>@conn3</p>
```

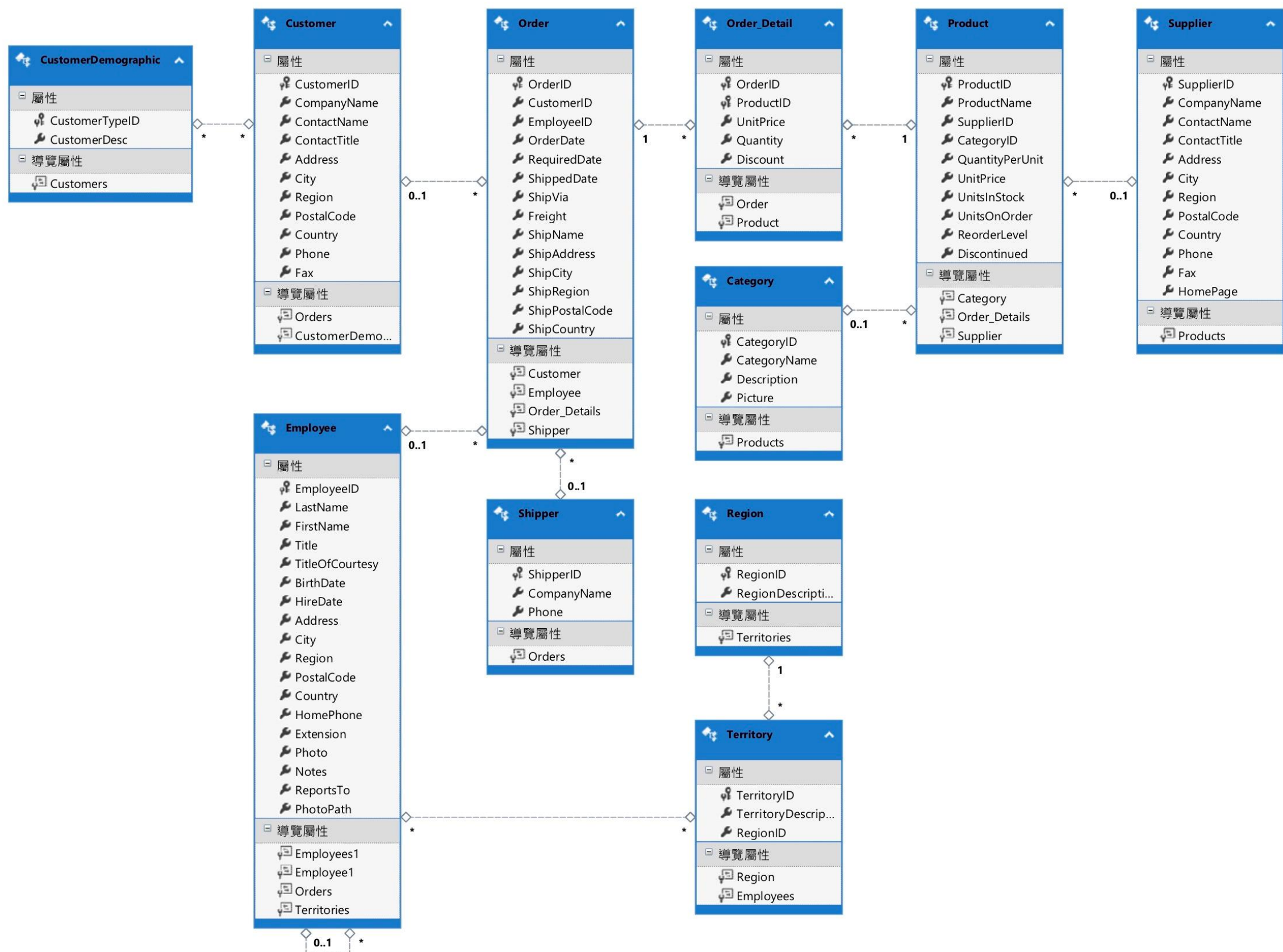
```
<p>@conn4</p>
```

```
<p>@conn5</p>
```


用Code First對既有資料庫 Scaffolding出DbContext及模型檔

15-4

Northwind 資料庫關聯



EF Core存取資料庫四要素

- Data Models 模型類別檔
- NorthwindContext (繼承 DbContext)
- appsettings.json資料庫連線設定
- 在DI Container註冊NorthwindContext類別

Scaffolding命令 – NuGet主控台

```
Scaffold-DbContext "Data Source=(localdb)\mssqllocaldb;Database=Northwind;"  
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

直接指定資料庫連線字串

SQL Server 資料庫 Provider

輸出到 Models 資料夾

或用 User Secret 中的 NorthwindConnection 連線字串設定：

```
Scaffold-DbContext Name=NorthwindConnection ← User Secret 使用者祕密  
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```


Scaffolding命令 – 命令視窗

或在命令視窗中的專案路徑，用 `dotnet ef dbcontext scaffold` 執行 Scaffolding：

```
dotnet ef dbcontext scaffold "Data Source=(localdb)\mssqllocaldb;  
Database=Northwind;" Microsoft.EntityFrameworkCore.SqlServer -o Models
```

或用 User Secret 中的 NorthwindConnection 連線字串設定：

```
dotnet ef dbcontext scaffold Name=NorthwindConnection ← User Secret 使用者祕密  
Microsoft.EntityFrameworkCore.SqlServer -o Models
```


Scaffolding指定資料表

- ✦ 在 NuGet 套件管理器主控台 (Package Manager Console)

以下是對八個資料表產出對映 Model 模型的命令：

```
Scaffold-DbContext Name=NorthwindConnection  
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models  
-tables Orders,"Order Details", Products, Categories, Suppliers,  
Employees, Customers, Shippers
```

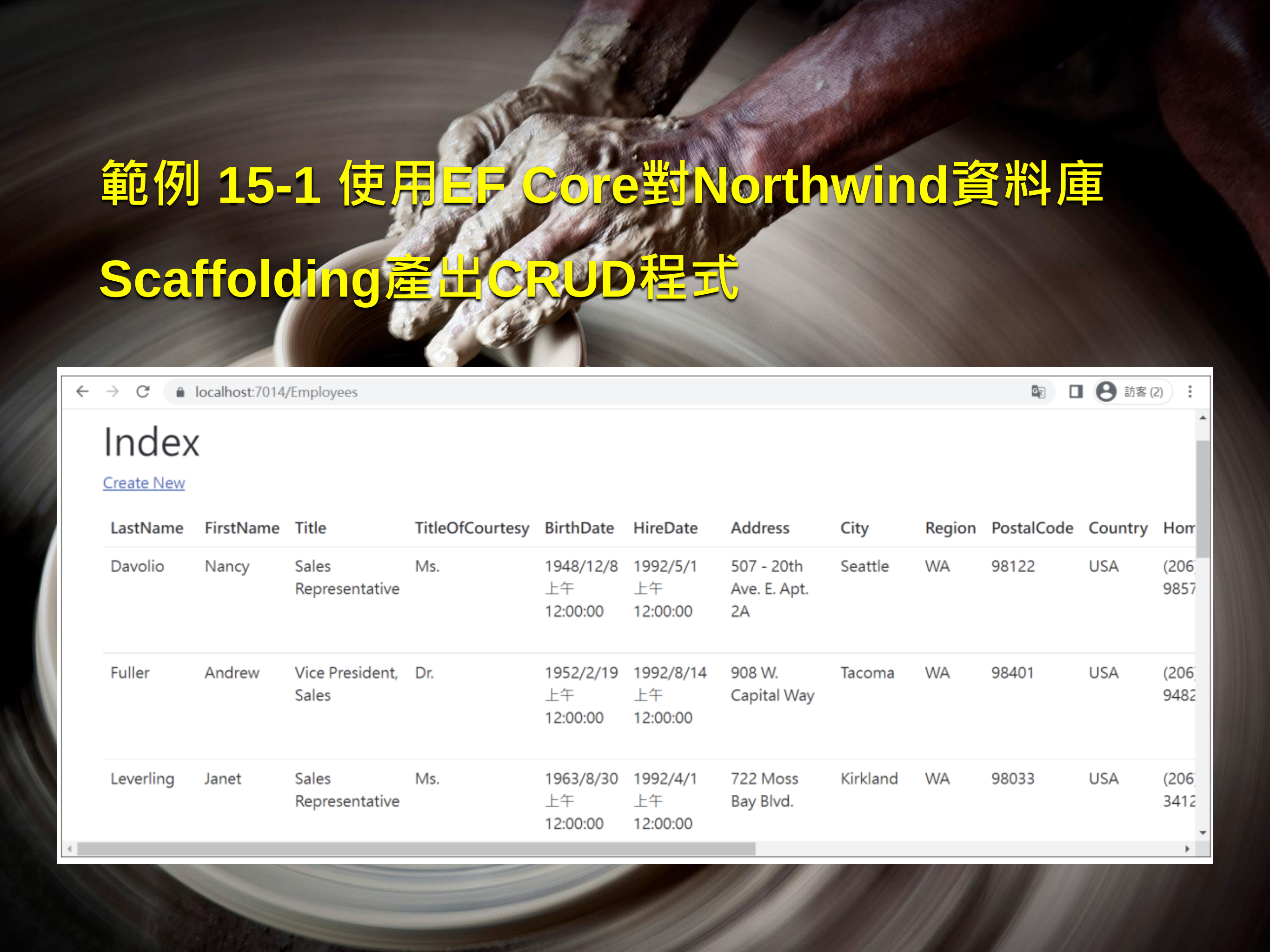
資料表以逗號分隔

- ✦ 在命令視窗執行 CLI 命令：

```
dotnet ef dbcontext scaffold Name=NorthwindConnection  
Microsoft.EntityFrameworkCore.SqlServer -o Models  
-t Orders -t "Order Details" -t Products -t Categories  
-t Suppliers -t Employees -t Customers -t Shippers
```

每個資料表皆需
-t 參數

範例 15-1 使用EF Core對Northwind資料庫 Scaffolding產出CRUD程式



← → ↻ localhost:7014/Employees 訪客 (2)

Index

[Create New](#)

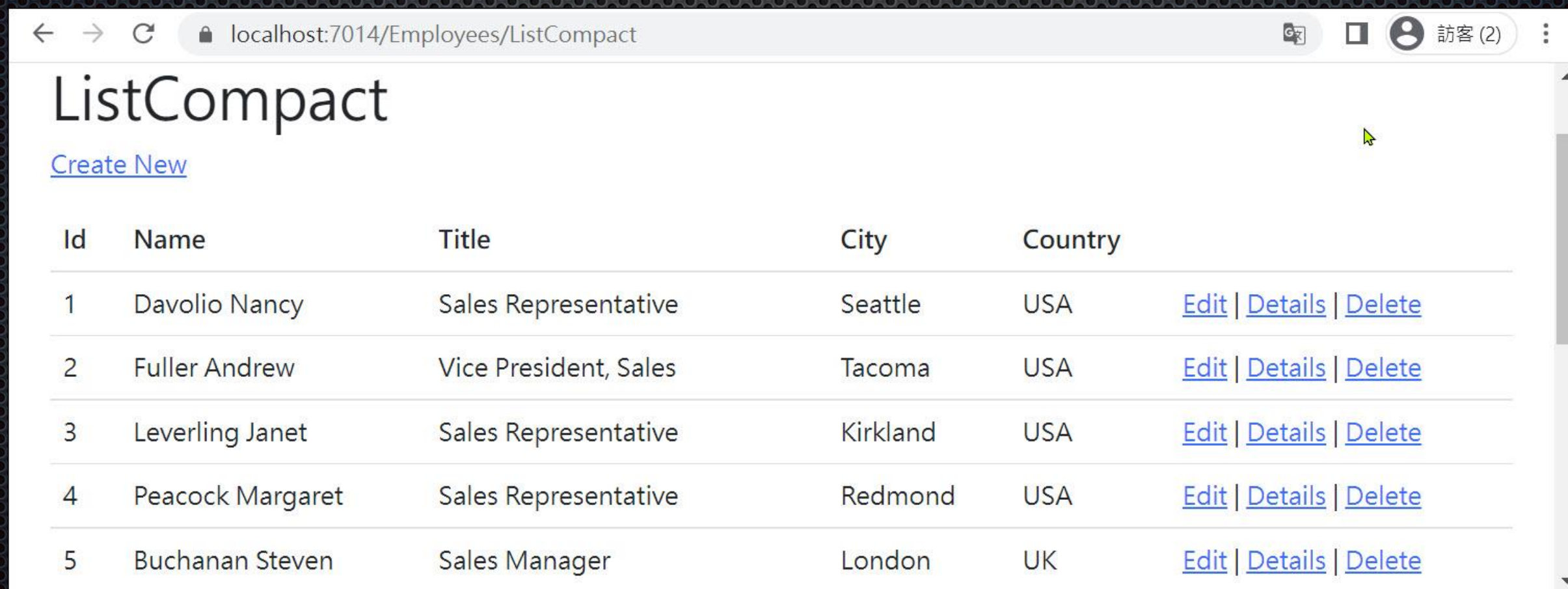
LastName	FirstName	Title	TitleOfCourtesy	BirthDate	HireDate	Address	City	Region	PostalCode	Country	HomePhone
Davolio	Nancy	Sales Representative	Ms.	1948/12/8 上午 12:00:00	1992/5/1 上午 12:00:00	507 - 20th Ave. E. Apt. 2A	Seattle	WA	98122	USA	(206) 985-9857
Fuller	Andrew	Vice President, Sales	Dr.	1952/2/19 上午 12:00:00	1992/8/14 上午 12:00:00	908 W. Capital Way	Tacoma	WA	98401	USA	(206) 948-9482
Leverling	Janet	Sales Representative	Ms.	1963/8/30 上午 12:00:00	1992/4/1 上午 12:00:00	722 Moss Bay Blvd.	Kirkland	WA	98033	USA	(206) 341-3412

Scaffolding程式樣板的問題

- Web顯示的資料欄位過多，影響版面
- SQL Server資料庫讀取過多的欄位，影響資料庫效能
- MVC程式耗費過多的CPU和記憶體處理及保存資料
- 影響網路傳輸及瀏覽器前端效能

改善方式 → 精簡查詢及顯示欄位

- LINQ只查詢必要的欄位
- 須配合View Model的建立
- View亦只顯示必要的欄位



The screenshot shows a web browser window with the address bar displaying 'localhost:7014/Employees/ListCompact'. The page title is 'ListCompact'. Below the title is a link 'Create New'. The main content is a table with 5 rows of employee data. Each row has columns for Id, Name, Title, City, and Country, followed by a set of links: 'Edit | Details | Delete'.

Id	Name	Title	City	Country	
1	Davolio Nancy	Sales Representative	Seattle	USA	Edit Details Delete
2	Fuller Andrew	Vice President, Sales	Tacoma	USA	Edit Details Delete
3	Leverling Janet	Sales Representative	Kirkland	USA	Edit Details Delete
4	Peacock Margaret	Sales Representative	Redmond	USA	Edit Details Delete
5	Buchanan Steven	Sales Manager	London	UK	Edit Details Delete

建立EmployeeViewModel

- 只讀取資料庫5個欄位並顯示，以增加效能
- 用View Model承接Data Model資料，並提供給LINQ語法作查詢

```
public class EmployeeViewModel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Title { get; set; }
    public string City { get; set; }
    public string Country { get; set; }
}
```

ViewModels/EmployeeViewModels.cs

1.LINQ to Entity - Query Syntax 查詢語法

```
public async Task<IActionResult> ListCompact()
{
    //1.LINQ to Entity - Query Syntax查詢語法
    var employees = await (from emp in _context.Employees
                           select new EmployeeViewModel
                           {
                               Id = emp.EmployeeId,
                               Name = emp.LastName + " " +
                                    emp.FirstName,
                               Country = emp.Country,
                               City = emp.City,
                               Title = emp.Title
                           }).ToListAsync();

    return View(employees);
}
```

EmployeesControllers.cs – ListCompact()

2.Method Syntax 方法語法

```
var employeesList = await _context.Employees
    .Select(x => new EmployeeViewModel {
        Id = emp.EmployeeId,
        Name = emp.LastName + " " +
            emp.FirstName,
        Country = emp.Country,
        City = emp.City,
        Title = emp.Title
    })
    .ToListAsync();
```

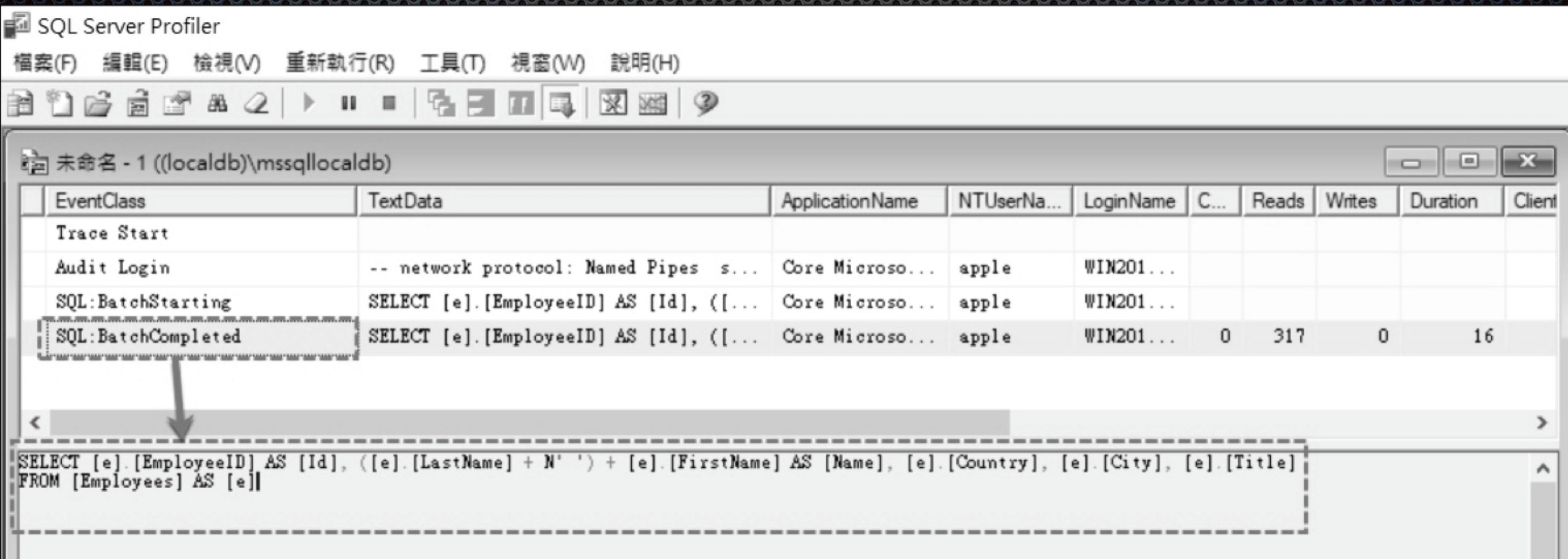

3. 使用FromSql()送出原生SQL查詢字串

原生SQL語法

```
var emps = await _context.Employees
    .FromSql($"Select * from dbo.Employees")
    .Select(x => new EmployeeViewModel
    {
        Id = x.EmployeeId,
        Name = x.LastName + " " + x.FirstName,
        City = x.City,
        Country = x.Country,
        Title = x.Title
    }).ToListAsync();
```


用 SQL Server Profiler 追蹤 SQL 查詢命令

- 檢查SQL是否僅查詢五個欄位



SQL Server Profiler

檔案(F) 編輯(E) 檢視(V) 重新執行(R) 工具(T) 視窗(W) 說明(H)

未命名 - 1 ((localdb)\mssqllocaldb)

EventClass	TextData	ApplicationName	NTUserNa...	LoginName	C...	Reads	Writes	Duration	Client
Trace Start									
Audit Login	-- network protocol: Named Pipes s...	Core Microso...	apple	WIN201...					
SQL:BatchStarting	SELECT [e].[EmployeeID] AS [Id], ([...	Core Microso...	apple	WIN201...					
SQL:BatchCompleted	SELECT [e].[EmployeeID] AS [Id], ([...	Core Microso...	apple	WIN201...	0	317	0	16	

SELECT [e].[EmployeeID] AS [Id], ([e].[LastName] + N' ') + [e].[FirstName] AS [Name], [e].[Country], [e].[City], [e].[Title]
FROM [Employees] AS [e]

Entity Framework Core

查詢資料庫常用語法

15-5

無條件查詢所有資料

15-5-1

無條件查詢所有資料

```
//查詢所有資料
public async Task<IActionResult> QueryAllData()
{
    //非同步
    List<Products> products = await _context.Products.ToListAsync();
    var employees = await _context.Employees.ToListAsync();

    //同步
    List<Orders> orders = _context.Orders.ToList();
    var orderDetails = _context.OrderDetails.ToList();

    return Ok("Finished.");
}
```

ProductsController.cs

用 First 和 Single 方法 查詢單一筆資料

15-5-2

First與Single方法

- **First方法**：傳回序列的第一個項目，或傳回序列中符合指定條件的第一個元素。若傳回序列無資料，會擲回例外狀況
- **FirstOrDefault方法**：傳回序列的第一個項目，或傳回序列中符合指定條件的第一個元素。若傳回序列無資料，則回傳預設值
- **Single方法**：傳回序列中符合指定條件之單一特定項目。若序列中有多個項目，就會擲回例外狀況
- **SingleOrDefault方法**：傳回序列的單一特定項目，若找不到該項目，則傳回預設值。如果序列中有多個項目，就會擲回例外狀況

First與Single方法比較

表 15-1 First 和 Single 方法比較

方法	傳回序列	傳回序列無資料	序列有多個項目
First	第一個項目	擲回例外狀況	-
FirstOrDefault	第一個項目	回傳預設值	-
Single	單一特定項目	擲回例外狀況	擲回例外狀況
SingleOrDefault	單一特定項目	回傳預設值	擲回例外狀況

查詢單一筆資料

```
public async Task<IActionResult> QuerySingleData(int Id = 1)
{
    //非同步
    var p1 = await _context.Products.FindAsync(Id);
    var p2 = await _context.Products.FirstAsync();
    var p3 = await _context.Products.FirstOrDefaultAsync();
    var p4 = await _context.Products.SingleAsync(p => p.ProductId == 1);
    var p5 = await _context.Products.SingleOrDefaultAsync(p => p.ProductId == 1);

    //同步
    var p6 = _context.Products.Find(Id);
    var p7 = _context.Products.First();
    var p8 = _context.Products.FirstOrDefault();
    var p9 = _context.Products.Single(p => p.ProductId == 1);
    var p10 = _context.Products.SingleOrDefault(p => p.ProductId == 1);

    return Ok("Finished.");
}
```


First與FirstOrDefault方法比較

- 回傳序列有一或多筆資料，First 與 FirstOrDefault 方法皆回傳第一筆
- 回傳序列若無資料，First 方法會擲出 `InvalidOperationException` 例外，但 `FirstOrDefault` 方法會回傳預設值

Single與SingleOrDefault方法比較

- 回傳序列若有多筆資料，Single 與 SingleOrDefault 方法皆會擲出InvalidOperationException 例外
- 回傳序列若無資料，Single 方法會擲出 InvalidOperationException例外，但 SingleOrDefault 方法會回傳預設值

以特定條件查詢資料

15-5-3

在Where中以特定條件查詢、排序

- 同時列出 Query Syntax 與 Method Query 兩種查詢語法

//以條件式過濾資料

```
public async Task<IActionResult> FilteringData()
```

```
{
```

//非同步

//Query Syntax查詢語法

```
var p1 = await (from p in _context.Products
                where p.UnitPrice >= 10 && p.UnitPrice <= 15
                orderby p.ProductName, p.UnitPrice
                select p)
                .ToListAsync();
```

Query Syntax查詢語法

//Method Syntax方法語法

```
var p2 = await _context.Products
                .Where(p => p.UnitPrice >= 10 && p.UnitPrice <= 15)
                .OrderBy(p => p.ProductName).ThenBy(p => p.UnitPrice)
                .ToListAsync();
```

Method Syntax方法語法

//同步

```
var p3 = (from p in _context.Products
          where p.UnitPrice >= 10 && p.UnitPrice <= 15
          orderby p.ProductName descending, p.UnitPrice ascending
          select p)
          .ToList();
```

升幂排序

降幂排序

```
var p4 = (_context.Products
          .AsEnumerable()
          .Where(p => p.UnitPrice >= 10 && p.UnitPrice <= 15))
          .OrderByDescending(p => p.ProductName).ThenByDescending(p => p.UnitPrice)
          .ToList();
```

轉為IEnumerable<Product>

```
var p5 = (_context.Products
          .AsQueryable()
          .Where(p => p.UnitPrice >= 10 && p.UnitPrice <= 15))
          .OrderByDescending(p => p.ProductName).ThenByDescending(p => p.UnitPrice)
          .ToList();
```

轉為IQueryable<Product>

```
return Ok("Finished.");
```

```
}
```


多個資料表的 Inner Join 查詢

15-5-4

兩個資料表的Inner Join查詢

//兩個資料來源- anonymous 匿名型別

```
var innerJoin1 =  
    from cate in _context.Categories  
    join prod in _context.Products on  
        cate.CategoryId equals prod.CategoryId  
    select new { Name = prod.ProductName,  
        Category = cate.CategoryName };
```

//兩個資料來源- anonymous 匿名型別

```
var innerJoin2 =  
    from cate in _context.Set<Categories>()  
    join prod in _context.Set<Products>() on  
        cate.CategoryId equals prod.CategoryId  
    select new { Name = prod.ProductName,  
        Category = cate.CategoryId };
```


三個資料表的Inner Join查詢

//3.三個資料來源 - 使用OrdersViewModel強型別

```
var innerJoin3 =  
from orders in _context.Orders.TagWith("3 Tables inner join")  
join odetails in _context.OrderDetails on orders.OrderId equals odetails.OrderId  
join prods in _context.Products on odetails.ProductId equals prods.ProductId  
    select new OrdersViewModel {  
    ProductId = odetails.ProductId, ProductName = prods.ProductName,  
    OrderId = orders.OrderId, UnitPrice = odetails.UnitPrice,  
    OrderDate = orders.OrderDate, ShipAddress = orders.ShipAddress,  
    ShipCity = orders.ShipCity, ShipCountry = orders.ShipCity };
```


Skip 與 Take 方法

15-5-5

Skip與Take方法

- Skip 與 Take方法可謂是「弱水三千，只取一瓢飲」
- Skip方法是跳過前幾筆資料，例如Skip(5)跳過前5筆
- 而 Take 方法是取幾筆資料，如 Take(10)只取 10 筆
- 二者還能結合使用，Skip(5).Take(10)是跳過前5筆，然後取 10 筆資料，這在資料分頁查詢時會用到

Skip與Take方法實例

```
public async Task<IActionResult> SkipTake()
{
    var products =
        from p in _context.Products
        where p.UnitPrice >= 10 && p.UnitPrice <= 15
        orderby p.ProductName descending, p.UnitPrice ascending
        select p;

    //Skip(5)跳過前5筆，然後Take(10)取10筆
    var query = await products.Skip(5).Take(10).ToListAsync();

    return Ok("Finished.");
}
```


IQueryable<T> vs. IEnumerable<T>
vs. ToList()

15-5-6

三種回傳型別

在 LINQ 或 LINQ to Entity 語法中，常會見到不同案例使用 `IQueryable<T>` vs. `IEnumerable<T>` vs. `ToList()` 三種回傳型別

- **`IQueryable<T>`**：Lazy Loading 延後執行，盡可能將 Expression 轉換成完整伺服器端 SQL 語法，僅回傳必要結果到記憶體
- **`IEnumerable<T>`**：Lazy Loading 延後執行，將資料全部載入記憶體後，再執行後續的操作
- **`ToList()`**：立即執行，在記憶體中建立 `List<T>` 集合物件


```
public IActionResult QueryType()
{
    //1.Lazy Loading 延後執行，盡可能將 Expression 轉換成完整伺服器端 SQL 語法，僅回傳必要結果到記憶體
    IQueryable<Products> products = _context.Products.TagWith("IQueryable")
        .Where(p => p.UnitPrice > 10);

    foreach (var item in products)
    {
        var i = item;
    }

    //2.Lazy Loading 延後執行，將資料全部載入記憶體後，再執行後續的操作
    IEnumerable<Products> prods = _context.Products.TagWith("IEnumerable")
        .AsEnumerable()
        .Where(p => p.UnitPrice > 10);

    foreach (var item in prods)
    {
        var i = item;
    }

    //3.立即執行，在記憶體中建立 List<T>集合物件
    List<Products> prodList = _context.Products.TagWith("ToList()")
        .Where(p => p.UnitPrice > 10)
        .ToList();

    return Ok("Finished.");
}
```


轉換成實際的SQL查詢語法

--IQueryable<Products>

```
SELECT [p].[ProductID], [p].[CategoryID], [p].[Discontinued], [p].[ProductName],  
[p].[QuantityPerUnit], [p].[ReorderLevel], [p].[SupplierID], [p].[UnitPrice],  
[p].[UnitsInStock], [p].[UnitsOnOrder]  
FROM [Products] AS [p]
```

WHERE [p].[UnitPrice] > 10.0

← 有 Where 條件式

--IEnumerable<Products>

```
SELECT [p].[ProductID], [p].[CategoryID], [p].[Discontinued], [p].[ProductName],  
[p].[QuantityPerUnit], [p].[ReorderLevel], [p].[SupplierID], [p].[UnitPrice],  
[p].[UnitsInStock], [p].[UnitsOnOrder]  
FROM [Products] AS [p]
```

↑ 缺乏 Where 條件式，查詢全部資料

-- ToList() 方法

```
SELECT [p].[ProductID], [p].[CategoryID], [p].[Discontinued], [p].[ProductName],  
[p].[QuantityPerUnit], [p].[ReorderLevel], [p].[SupplierID], [p].[UnitPrice],  
[p].[UnitsInStock], [p].[UnitsOnOrder]  
FROM [Products] AS [p]
```

WHERE [p].[UnitPrice] > 10.0

← 有 Where 條件式

IEnumerable<T>回傳型別缺乏 Where條件式

- 三者實際送出的SQL查詢語法，其中
IEnumerable<Products>的SQL語句缺少Where條件式，便會查詢所有資料，在資料庫端查詢成本和效能是比較吃重的

使用原生SQL查詢

15-5-7

使用FromSqlRaw()送出原生SQL查詢字串(1)

//1. 使用FromSqlRaw()送出原生SQL查詢字串

```
var productsSql =  
    await _context.Products  
        .FromSqlRaw("Select * from dbo.Products")  
        .Select(p =>  
            new ProductViewModel {  
                Id = p.ProductId,  
                Name = p.ProductName,  
                UnitPrice = p.UnitPrice,  
                UnitsInStock = p.UnitsInStock,  
                UnitsOnOrder = p.UnitsOnOrder })  
        .AsNoTracking()  
        .ToListAsync();
```

送出原生SQL查詢字串

不追蹤效能較佳

//2. 呼叫GetTopSales預存程序

```
var productsSP = _context.Products  
    .FromSqlRaw("EXECUTE dbo.GetTopSales")  
    .ToList();
```

呼叫GetTopSales預存程序

使用FromSqlRaw()送出原生SQL查詢字串(2)

//3.呼叫GetTopSalesForCategory預存程序，並傳入參數

```
int categoryId = 1;
```

```
var prodsPamam1 = _context.Products
```

```
    .FromSqlRaw("EXECUTE dbo.GetTopSalesForCategory {0}",  
        categoryId).ToList();
```

傳遞categoryId給{0}

//4.FromSqlInterpolated插入字串

```
var prodsPamam2 = _context.Products
```

```
    .FromSqlInterpolated($"EXECUTE dbo.GetTopSalesForCategory  
        {categoryId}").ToList();
```

傳遞categoryId參數

補充說明

- 絕對不要將未驗證的字串或插入字串（\$""）傳遞至 FromSqlRaw 或 ExecuteSqlRaw 方法中，避免資料庫攻擊
- FromSqlInterpolated 和 ExecuteSqlInterpolated 方法允許使用字串內插補點語法，以防止 SQL 插入式攻擊的方式
- 資料若僅作顯示用途，用 AsNoTracking() 方法不追蹤查詢，可獲得更高效能
- 至於追蹤的用途在於對 Entity 做變更異動，呼叫 SaveChanges() 或 SaveChangesAsync() 方法後，會將異動儲存至資料庫

資料庫交易程式

15-6

交易(Transaction)

交易(Transaction)是指將一連串的工作視為一個邏輯單元 (Logic Unit)，而交易本身必定具備 ACID 特性

- 不可部份完成性 (**A**tomicity)
- 一致性 (**C**onsistency)
- 隔離性 (**I**solation)
- 持久性 (**D**urability)

不可部份完成性(Atomicity)

- 必須將交易程式中所有的工作項目全部完成，才算是一個完整交易
- 否則假設交易單元中有 100 個工作項目，即便有 99 項完成，1 項失敗，都算交易失敗

一致性(Consistency)

- 交易完成時，全部的資料必須維持一致性的狀態
- 在關聯式資料庫 (Relational Database) 中，必須將所有的規則 (Rule) 套用的修改，以維護所有的資料整合性 (Integrity)
- 所有的內部資料結構，例如B型樹狀結構索引(B-tree Index)或是雙向連結串列(Doubly-Linked List)，在交易終止時必須是正確的

隔離性(Isolation)

- 並行的交易所做的修改，必須與其他任何並行的交易所做的修改隔離
- 交易所辨識的資料，不是處於另一筆並行的交易修改資料之前的狀態，就是處於第二筆交易完成後的狀態，但是卻無法辨識中繼狀態
- 這稱為序列化能力（Serializability），因為這樣可以產生重新載入起始資料，並重新執行一系列的交易所做的修改，以便讓資料最終能夠與原始交易所做的修改執行後的狀態相同的能力

持久性(Durability)

- 交易完成之後，其作用便永遠存在於系統之中。
- 即使系統發生失敗的事件，但修改仍會保存

資料庫交易確保資料的嚴謹

- 如果您對資料庫程式的嚴謹性要求很高時，程式就應融入交易機制，以便確保任何非預期的狀況發生時，資料交易可以百分之百的完成
- 若無法完成時，也要能夠回復到交易前的資料狀態，等待下一次再進行資料庫交易
- 而不是完成一半，但哪些項目完成、哪些項目沒完成都不清楚的爛攤子


```
using (var transaction = _context.Database.BeginTransaction())
{
    try
    {
        _context.Products.Add(new Products { ProductName = "Cola", UnitPrice = 1,
                                              UnitsInStock = 15, UnitsOnOrder = 200 });
        await _context.SaveChangesAsync();

        Products wine = new Products { ProductName = "Wine", UnitPrice = 20,
                                        UnitsInStock = 10, UnitsOnOrder = 50 };
        _context.Products.Add(wine);
        await _context.SaveChangesAsync();

        Products sugar = new Products { ProductName = "Sugar", UnitPrice = 2,
                                        UnitsInStock = 50, UnitsOnOrder = 250 };
        _context.Products.Add(sugar);
        await _context.SaveChangesAsync();

        _context.Remove(wine);
        await _context.SaveChangesAsync();

        var identityCurrent = await _context.Products.FromSqlRaw("select * from dbo.Products
                                                                where ProductId = IDENT_CURRENT('Products')").FirstOrDefaultAsync();

        transaction.Commit();

    }
    catch (Exception ex)
    {
        transaction.Rollback();
    }
}
```

以下新增三筆資料，再刪除一筆，整個視為一個交易

Commit() 確認交易成功

Rollback() 交易失敗回復

交易程式說明

- EF Core 是透過 DbContext.Database API 初始 Transaction 交易
- 上一頁在 Action 中使用交易，新增三筆資料，再刪除一筆，整個視為一個交易
- 上一頁程式呼叫四次 SaveChangesAsync()方法是刻意為之。因為一般呼叫 SaveChangesAsync()後，EF Core 會立即將異動資料寫入資料庫，但使用交易卻不會立即寫入

The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹**、**ASP.NET Core 3.x MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw